

SE4040 - Enterprise Application Development

Lab Sheet 9 – Enterprise Application Integration

Student NO: IT22555380

Student Name: Wijesinghe LP.P.

1. Purpose of the Lab

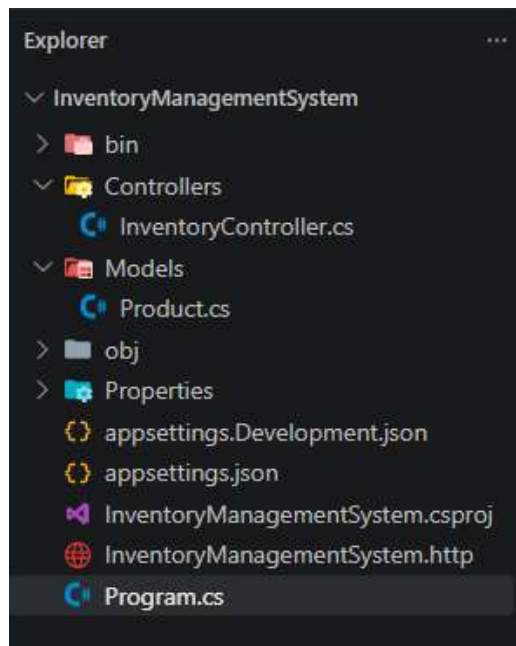
This lab demonstrates Enterprise Application Integration (EAI) by integrating two independent applications: an Order Management System and an Inventory Management System. The applications communicate using REST APIs, with JSON used as the shared data format. The Order Management System checks product availability through the Inventory Management System before accepting or rejecting an order.

2. Source Code Implementation

2.1 Inventory Management System

Endpoint: GET /api/inventory/{productName}

Project Structure:



Product.cs

```
namespace InventoryManagementSystem.Models
{
    public class Product
    {
        public int ProductId { get; set; }
        public string ProductName { get; set; } = string.Empty;
        public int Quantity { get; set; }
    }
}
```

InventoryController.cs

```
using Microsoft.AspNetCore.Mvc;
using InventoryManagementSystem.Models;

namespace InventoryManagementSystem.Controllers
{
    [ApiController]
    [Route("api/inventory")]
    public class InventoryController : ControllerBase
    {
        private static readonly List<Product> Products = new()
        {
            new Product
            {
                ProductId = 1,
                ProductName = "Laptop",
                Quantity = 15
            },
            new Product
            {
                ProductId = 2,
                ProductName = "Phone",
                Quantity = 10
            }
        };

        [HttpGet("{productName}")]
        public IActionResult GetInventory(string productName)
        {
            var product = Products.FirstOrDefault(p =>
                p.ProductName.Equals(
                    productName,
                    StringComparison.OrdinalIgnoreCase));

            if (product == null)
```

```

        {
            return NotFound(new
            {
                message = "Product not found"
            });
        }

        return Ok(new
        {
            productName = product.ProductName,
            quantity = product.Quantity,
            available = product.Quantity > 0
        });
    }
}
}

```

Program.cs

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddOpenApi();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.MapControllers();

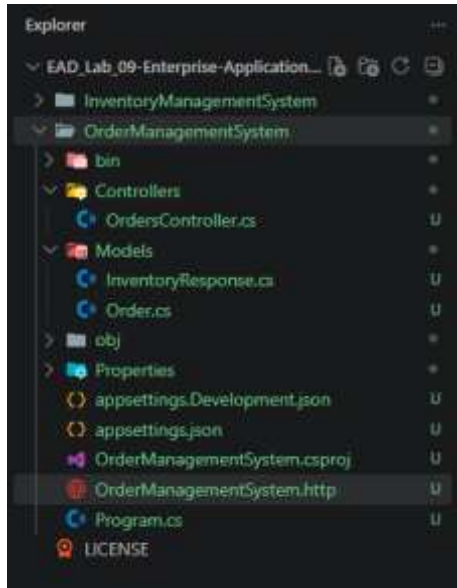
app.Run();

```

2.2 Order Management System

Endpoint: POST /api/orders

Project Structure:



Order.cs

```
namespace OrderManagementSystem.Models
{
    public class Order
    {
        public string CustomerName { get; set; } = string.Empty;
        public string Product { get; set; } = string.Empty;
        public int Quantity { get; set; }
    }
}
```

OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;
using OrderManagementSystem.Models;
using System.Net;

namespace OrderManagementSystem.Controllers
{
    [ApiController]
    [Route("api/orders")]
    public class OrdersController : ControllerBase
    {
        private readonly HttpClient _httpClient;
    }
}
```

```

public OrdersController(HttpClient httpClient)
{
    _httpClient = httpClient;
}

[HttpPost]
public async Task<IActionResult> PlaceOrder(Order order)
{
    var response = await _httpClient.GetAsync(
        $"http://localhost:5044/api/inventory/{order.Product}");

    if (response.StatusCode == HttpStatusCode.NotFound)
    {
        return NotFound(new
        {
            message = "Product not found"
        });
    }

    if (!response.IsSuccessStatusCode)
    {
        return StatusCode(500, new
        {
            message = "Inventory service unavailable"
        });
    }

    var inventory =
        await response.Content.ReadFromJsonAsync<InventoryResponse>();

    if (inventory == null)
    {
        return StatusCode(500, new
        {
            message = "Invalid inventory response"
        });
    }

    if (inventory.Available &&
        inventory.Quantity >= order.Quantity)
    {
        return Ok(new
        {
            message = "Order placed successfully"
        });
    }

    return BadRequest(new

```

```

        {
            message = "Insufficient stock"
        });
    }
}

```

Program.cs

```

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddHttpClient();

builder.Services.AddOpenApi();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.MapControllers();

app.Run();

```

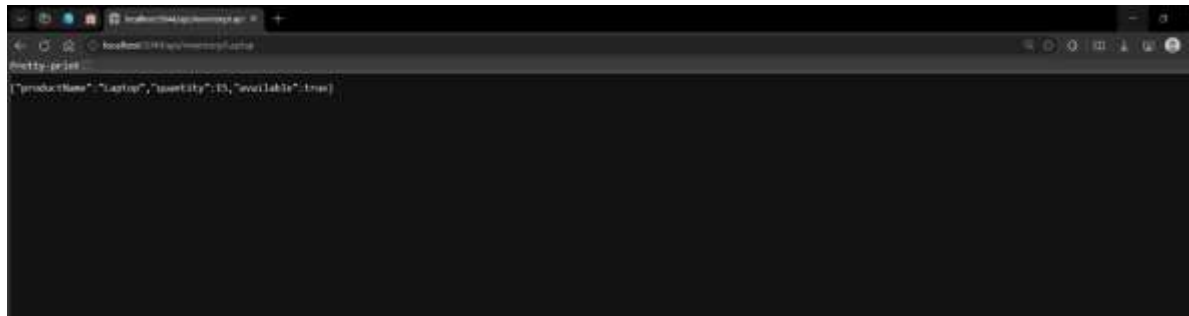
InventoryResponse.cs

```

namespace OrderManagementSystem.Models
{
    public class InventoryResponse
    {
        public string ProductName { get; set; } = string.Empty;
        public int Quantity { get; set; }
        public bool Available { get; set; }
    }
}

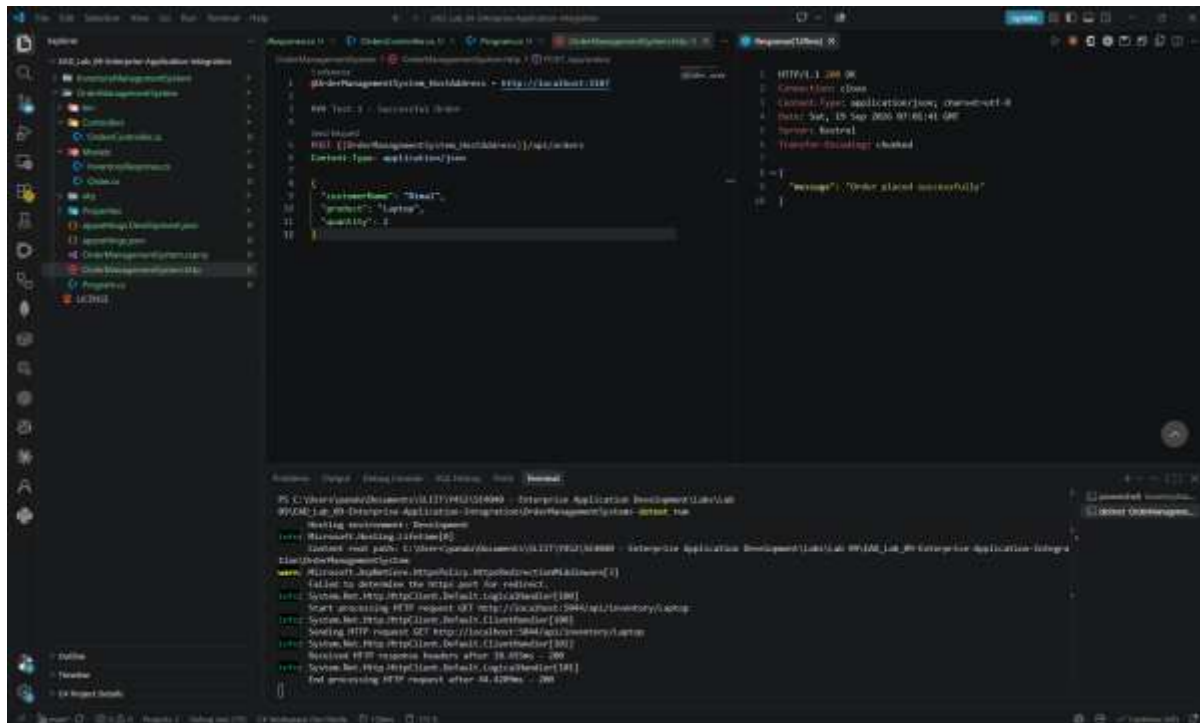
```

3. Inventory API Testing

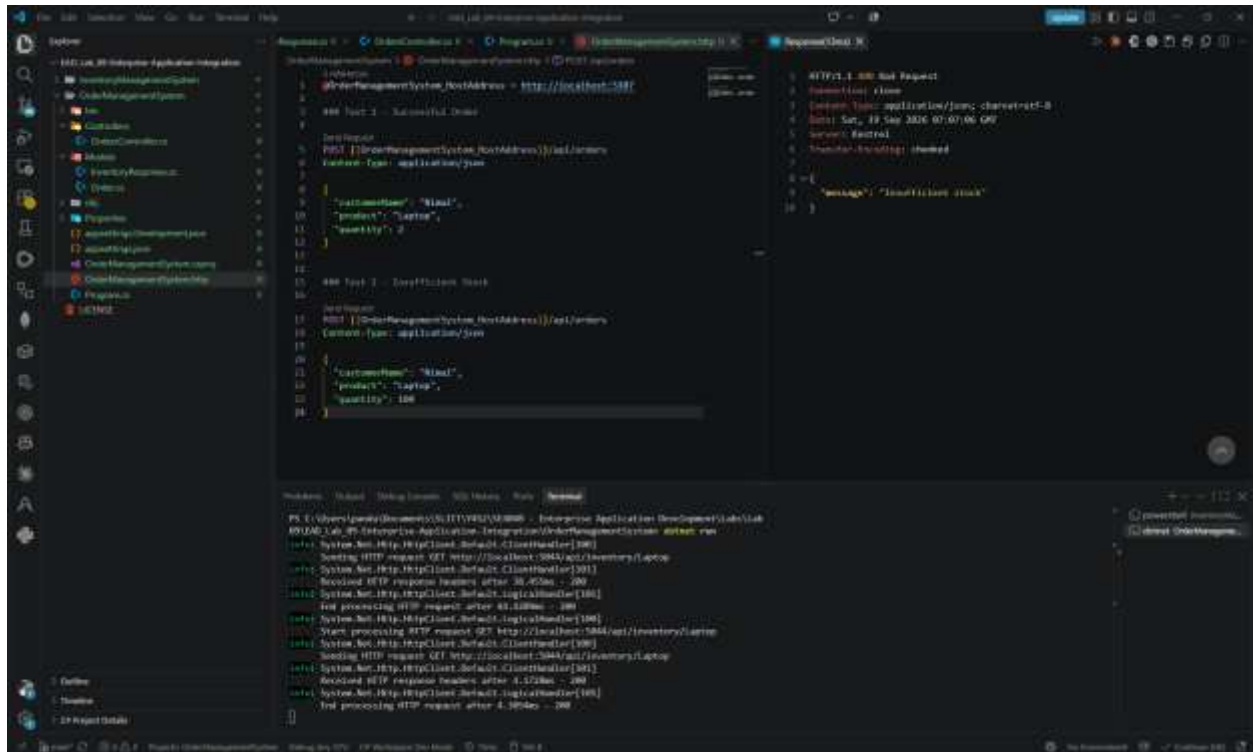


4. Order Management System Integration Testing

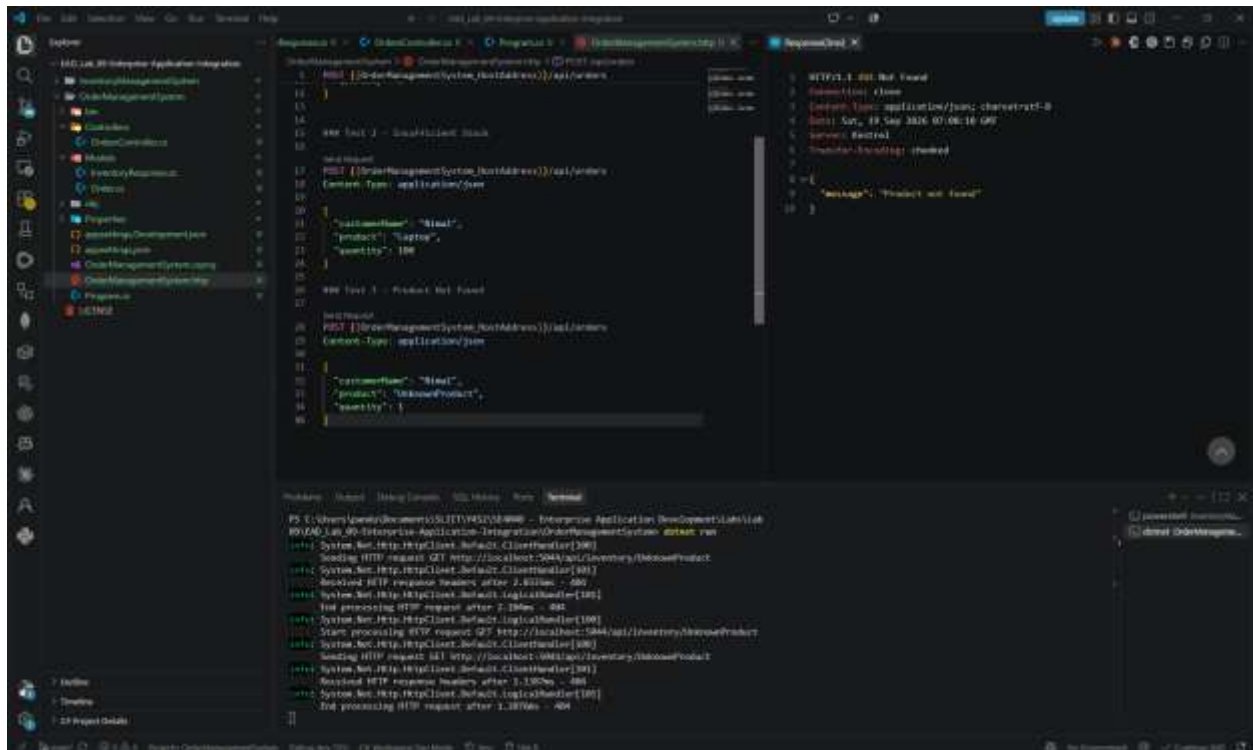
4.1 Successful Order



4.2 Rejected Order – Insufficient Stock



4.3 Product Not Found Test



5. Integration Architecture



6. Integration Analysis

1. What EAI approach is used in this lab?

Answer:

This lab uses application-level integration through direct REST API invocation. The Order Management System consumes the REST API exposed by the Inventory Management System.

2. What data format is exchanged between the two applications?

Answer:

JSON (JavaScript Object Notation) is used as the data format exchanged between the two applications.

3. Is this integration synchronous or asynchronous? Explain briefly.

Answer:

This integration is synchronous because the Order Management System waits for a response from the Inventory Management System before accepting or rejecting the order.

4. What happens if the Inventory Management System is unavailable?

Answer:

If the Inventory Management System is unavailable, the Order Management System cannot check product availability, so the order cannot be processed successfully.

5. State one advantage of this integration approach.

Answer:

One advantage is that product availability can be checked automatically in real time, reducing the need for manual checking.

6. State one disadvantage of this integration approach.

Answer:

One disadvantage is that the Order Management System depends on the Inventory Management System being available. If the Inventory API is unavailable, the order cannot be processed successfully.

7. How does this design create coupling between the two applications?

Answer:

This design creates coupling because the Order Management System directly depends on the Inventory Management System's REST API endpoint and response format.